PipeRewind

A Time-Travel Debugger for Shell Pipelines

Luiz Takahashi & Ehzoc Chavez

Abstract

Shell pipelines are incredibly powerful but famously opaque. Identifying the root cause of a failure in a long chain of commands often requires cumbersome workarounds like injecting `tee` to inspect intermediate data. This document explores **PipeRewind**, a time-travel debugger written in C that solves this problem. PipeRewind transparently interposes data streams, monitors process states, logs everything sequentially, and provides both interactive scrubbing capabilities (replay) and real-time live-attachment features via an `ncurses` terminal user interface.

Contents

1 What PipeRewind Really Is **2**

2 Why It Matters **2**

3 Core Capabilities **2**

4 Code Structure and Architecture **3**

5 How to Use PipeRewind **3**

5.1 Compilation 3

5.2 Execution Modes 4

5.3 TUI Hotkeys 4

1 What PipeRewind Really Is

PipeRewind is an extremely precise debugging framework specifically designed to observe, capture, and visualize the execution of standard UNIX shell pipelines.

When a user strings together a command sequence such as `cmd A | cmd B | cmd C`, PipeRewind sits silently between the interconnected `stdout` and `stdin` file descriptors of each process. It intercepts every byte of data passing through the system, measures the exact time standard events occur (like process creation, sleeping, and dying), and records these transactions into a proprietary binary `.prt` (PipeRewind Trace) trace file. Waitings, blockages, and textual outputs are indexed seamlessly.

Finally, a dedicated graphical `ncurses` application enables engineers to playback, scrub back-and-forth in time, and view comparative differences in the transmitted data payload.

2 Why It Matters

The standard philosophy of Unix operates on text streams running linearly between disparate utility binaries. While this flexibility allows amazing combinations, it introduces catastrophic opacity. When a pipeline fails (e.g., outputs zero lines, or crashes), tracing *where* exactly the failure originated is notoriously difficult. Was `cmd A` producing malformed inputs, or did `cmd B` incorrectly parse them?

Traditional Solutions Fall Short:

- **`strace`:** Too low-level and noisy; sifting through thousands of system calls across multiple heavily forked processes is mentally exhausting.
- **`tee`:** Manually restructuring the command strings with `tee ./dump1.txt | cmd B | tee ./dump2.txt` litters the filesystem, requires modifying the source scripts, and loses timing information entirely.

PipeRewind solves this. Instead of retrofitting debuggers or modifying scripts, you inject `piperewind.c` exactly once and immediately get an immutable history map of not just the variables, but the actual textual data passed along dynamically.

3 Core Capabilities

PipeRewind elevates typical shell monitoring. The major capabilities incorporated include:

1. **Transparent High-Speed Interception:** Using Linux `epoll()` capabilities to monitor descriptors asynchronously, recording throughput efficiently with unnoticeable CPU overhead.
2. **Process State Polling:** Tracking `/proc/[pid]/status` in the background to discern if a process is `RUNNING`, `BLOCKED`, or `EXITED`.
3. **Side-by-Side Diff Panel:** Comparing `DATA_OUT` from Stage N against `DATA_IN` to Stage N+1, visualizing exactly what payload transitioned downstream via a Longest-Common-Subsequence algorithm.

4. **Windowed Trace Loading:** Processing massive pipeline topologies by implementing a smart memory sliding-window mechanism over trace logs—providing fast scrubbing capabilities without OOM (out-of-memory) crashes.

5. **Live-Attach Mode:** Spawning the terminal UI interface immediately while executing the pipeline concurrently, trailing process data as it enters dynamically using synchronized daemon threading.

4 Code Structure and Architecture

The project relies exclusively on C99, structured systematically around UNIX system methodologies:

- **`src/pipeline.c`:** A state-machine parser. Rather than naive string-splitting, it interprets UNIX shell tokens natively (processing single/double escaping constraints perfectly) before executing commands.
- **`src/capture.c`:** The interception engine. Invokes processes, handles anonymous `dup2()` pipe bridging, and reacts asynchronously to buffer dispatches via `epoll`.
- **`src/procstate.c`:** Context-aware telemetry module reading localized Kernel `/proc` values inside tight feedback loops to identify schedule blocking anomalies.
- **`src/trace.c`:** Handles writing and reading data onto disk format. Encapsulates dynamic index capacity expansion so queries like "give me data belonging to stage N at timestamp T" map directly to disk payload jumps natively.
- **`src/tui.c` & `src/diff.c`:** The graphical component. Computes window timelines, renders events sequentially over visual buffers, manages asynchronous pthreads binding 'Live' pipelines to updates, and dynamically LCS diffs consecutive payload fragments.

5 How to Use PipeRewind

Using PipeRewind follows intuitive syntax mimicking standard Unix command-line execution interfaces.

5.1 Compilation

The project incorporates a basic Makefile. Ensure you have GCC and `libncurses-dev` dependencies satisfied natively.

```
# Simply compile all modules locally
make clean && make
```

5.2 Execution Modes

PipeRewind allows multi-stage operation.

- **`record`:** Creates a headless capturing session and outputs data directly onto the disk (defaults to `trace.prt`).

```
./piperewind record "cat /var/log/syslog | grep error | sort"
```

- **`replay`:** Summons the visual graphical UI (TUI) overlay rendering pipeline data streams out over time natively.

```
./piperewind replay trace.prt
```

- **`live`:** Instantiates real-time debugging. The capture engine spins up as an isolated thread in the background, drawing timeline updates transparently back into the TUI frame utilizing follow/auto-scroll heuristics.

```
./piperewind live "tail -f /var/log/nginx/access.log | jq ."
```

5.3 TUI Hotkeys

While inside the `ncurses` view:

- **Up/Down Arrows:** Shift selection highlight across interacting stages.
- **j / k:** Frame-step timeline indexes forward or backwards in time.
- **J / K:** Fast scrubbing (10 index gaps).
- **d:** Toggles the Side-by-Side Diff interface analyzing data modifications between parallel downstream buffers directly.
- **f:** Toggles timeline "Follow Mode" binding navigation automatically to late-joining live daemon updates.
- **q:** Exit the trace interface confidently.